

# NeuroSymbolic Marketplace — Product Brief & Protocol Roadmap

2026-08-20 · Status: Prototype 4 is live on Base mainnet (*mainnet-base*), testing assumptions. Some mechanisms are deliberately not in it yet — they ship in the order this roadmap sets out. Part 1 describes the product that can deploy without changing protocol tokenomics. Part 2 covers what does change tokenomics, and therefore goes through an RFC and governance, not a quiet deploy.

---

## Terms used (DKG V10 vocabulary)

- **DKG node** — the node software from OriginTrail/dkg; runs in edge or core role, with a dashboard UI, CLI, and HTTP API.
  - **Knowledge Asset (KA)** — a unit of published knowledge: RDF statements plus a Merkle proof, anchored on-chain. Publishing costs TRAC.
  - **Context Graph (CG)** — a scoped knowledge domain (the node UI calls these "projects"), with open or curated access.
  - **Shared Working Memory (SWM)** — the gossip layer between Context Graph peers. "The lane" below means marketplace traffic carried over SWM.
  - **Verifiable Memory (VM)** — the permanent, on-chain memory layer that publishing writes into.
  - **Epoch** — the network's reward period.
  - **Ask** — the price a node sets for its own service. Today a node sets an on-chain ask for publishing (TRAC per KB-epoch); this product adds asks for inference (per token) and queries (per returned result).
  - **Allowance / ceiling** — how many tokens a plan lets a buyer spend, per model, per period.
  - **Statement** — the once-per-period usage total that both sides sign.
  - **PCA** — Publisher Conviction Account: today's prepaid TRAC pool for publishing.
  - **Delegated staking / operator fee** — TRAC holders stake to nodes they trust (StakingV10); node operators take a fee from the rewards.
- 

## Part 1 • Product Brief

### One-liner

Every DKG node can buy and sell AI services — model inference and knowledge queries — priced in TRAC. Buyers pay through simple weekly or monthly plans with per-model allowances. Both sides count usage independently, and the counts are compared once per period. Using it should feel like any AI subscription, and finding models should feel like OpenRouter.

### The problem

Today's inference markets run on trust: the provider counts the tokens, and the buyer believes the bill. Decentralized alternatives flip the trust but lose the usability — custom APIs, wallet ceremony at every step, no familiar experience. Meanwhile, DKG nodes already hold the two things this market trades — knowledge and compute — but have no built-in way to sell serving them.

Our own prototypes added one more finding: **settling every small payment on-chain does not work at consumer scale**. In live runs, 3.0 TRAC of deposits were needed to bill 2,376  $\mu$ TRAC of actual usage, and the settlement checks correctly refused to pay out because network fees would have been  $\sim 1,800\times$  larger than the earnings. The measurement worked; the money movement was too heavy.

## What it is

A marketplace module inside the DKG node (a feature-flagged workspace package with a clean boundary). It gives every node three roles:

- **Seller** — offers two kinds of models:  *weights-pinned* local models (the model file is hashed, its tokenizer is pinned, served via llama.cpp) and  *upstream-claimed* resold subscriptions (OpenAI, DeepSeek, or any compatible upstream). Each offer is published to the DKG as a Knowledge Asset, with the node's ask per token (inference) or per returned result (queries). Every response the seller sends is signed.
- **Buyer** — checks the seller's signed quote, subscribes, and counts its own consumption in parallel against its per-model allowances. Once per period, the two counts are reconciled into one signed statement. Random spot-checks (re-counting one call in N with the pinned tokenizer) keep sellers honest at almost no cost. Failed or undelivered calls never reduce the allowance. The detailed per-call verification built in the prototypes remains available as the dispute path when a statement doesn't match.
- **Gateway** — one OpenAI-compatible API per node. The node's owner mints keys with budgets and scopes, so agents — co-located or remote, built on OpenClaw, Hermes, ElizaOS, or anything else — use marketplace models by changing a base URL and pasting a key. Two transports: direct HTTPS (with streaming) and **the lane** — requests and responses carried as gossiped messages over SWM, so a seller needs no public endpoint at all.

## UX north star

OpenRouter parity, then past it. Discovery works on two axes, with a clear hierarchy:

- **By model (primary)**. A catalog grouped by canonical Model KAs — logos, prices in both  $\mu$ TRAC and USD, and a table of every node offering that model. The router compares sellers within a model, so independent asks become competition instead of homework for the user.
- **By node (the trust layer)**. A page per node — everything it serves across / classes, its reputation, uptime, and statement-verified volume. This is where preferences live: pin trusted nodes, filter by provenance class, stay within your Context Graph community, or subscribe to one node's whole shelf.

One key and one endpoint no matter how many sellers exist — sellers are providers, never separate APIs. Independent pricing is handled by three rules: the router competes sellers within a model; a plan's ceiling is guaranteed at the buyer's **price cap** (you get at least the promised tokens, more if the router finds cheaper); and asks are **per-period commitments** — price changes take effect at the next cycle, aligned with epochs.

Day to day, the centerpiece is a familiar meter: "Qwen2.5 14B — 1.2M of 5M, resets in 12 days." Once per period, one statement line: "This period: our count 1,238,400 · provider count 1,238,400 ✓." Rankings use statement-verified volume, not self-reported traffic. Onboarding is two steps: Subscribe → Key. Measured so far: a fresh node reached its first completion in **55 seconds on mainnet, 24 on devnet**.

## How TRAC is spent — the mechanisms and what each needs

| Mechanism                          | Experience                                                                                                                                                                 | Settlement                                                                                                                                                  | Protocol change                                             |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| <b>Metered tabs</b><br>(retired)   | deposit → spend → exact refund; the prototypes' rail                                                                                                                       | proved the verification machinery on live mainnet; retired as too heavy (two on-chain transactions per engagement, capital locked at ~0.08% use)            | n/a — not in the product                                    |
| <b>Subscriptions</b><br>(the rail) | weekly or monthly TRAC payment → per-model token ceilings; no refunds; price frozen for the cycle; unused allowance expires at reset; at the cap: wait, upgrade, or top up | one on-chain payment per period, like publishing a KA; consumption reduces ceilings <b>off-chain</b> , counted by both sides and reconciled once per period | none at the app layer; epoch-pooled distribution is Part 2  |
| <b>PCA allowances</b>              | a holder-chosen share of a Conviction Account allowance is spendable on queries and inference; spent is spent                                                              | same rules as subscriptions                                                                                                                                 | <b>RFC — Part 2</b>                                         |
| <b>Fiat (Stripe)</b>               | recurring card payment → per-model entitlements; cancelling = not renewing; a chargeback revokes the remaining ceiling                                                     | a fiat gateway converts; TRAC settles underneath, always                                                                                                    | none (test-mode designed; production waits on legal review) |

Why no refunds? Because it makes a subscription the *same kind of object* as publishing a Knowledge Asset: value committed at one moment, rewarding the network over an epoch. It also changes what verification protects — not "money back" but "allowance preserved" — and it makes wash-trading strictly more expensive. **The chain footprint is the point: one transaction per buyer per period. Zero per message. Zero per statement.** Part 2 adds a single pooled distribution per epoch for the whole network.

## Evidence base

Four prototype iterations on Base mainnet with real TRAC and independent counterparties: the books balanced to the microTRAC from both sides; three funded engagements with exact refunds; a bill for undelivered work cancelled automatically on screen; 13 defects found, zero by end users; every claim backed by transcripts, ledger reads, and Basescan transactions — and every claim that turned out false

corrected in place, not silently replaced. The deposit-and-refund rail that produced this evidence retires with Prototype 4; the verification machinery it proved lives on at statement granularity.

## Explicitly out of the product track

Anything that changes protocol emissions or the node payout formula. That is Part 2.

---

## Part 2 • Protocol Roadmap — the RFC track

*These items are **not** in the next prototype, because they change tokenomics. They go through a dedicated RFC, community scrutiny, and governance.*

### The economic loop

Subscription money flows into the network → it is distributed to nodes that served, weighted by a new factor → delegated stake follows the nodes that earn → the operator fee turns that into operator income. The last step needs **no new protocol surface at all**: the operator fee already exists, delegators already compare nodes, and fee competition already keeps operators honest. New revenue simply flows through an economy that already works.

The headline property, and the RFC's core motivation: **stake migrates toward nodes that serve, so the network's security budget gradually lines up with its usefulness**. Publishing made *hosting* knowledge earn yield; this makes *serving* it — answering queries, running inference — earn yield too. Delegators become the mechanism that decides which nodes deserve to grow.

### Item 1 — The Query/Inference Factor

Add a service factor to the node payout formula, next to the publishing factor. Publishing rewards knowledge at rest (hosting, availability); the Query/Inference Factor rewards knowledge in motion (queries answered, inference served). It is measured in **statement-verified service volume** per node per epoch: both sides count, both sides sign **one statement per relationship per period** — not per call. Merkle roots over those statements feed the distribution. If a statement is contested, the prototypes' detailed per-call verification is the dispute path.

Design commitments carried into the RFC:

- **Closed loop.** Service rewards paid out in an epoch never exceed service payments that came in. Someone buying from themselves can get back at most their own money, minus everyone else's share.
- **Concentration discounts.** Volume from many distinct buyers counts more than the same volume from one buyer. Wash-trading also burns the attacker's own non-refundable allowances.
- **Stake coupling, concave.** The factor compounds with delegated stake enough to pull delegation toward serving nodes — but with diminishing returns, plus a small stake-independent part. A new GPU operator with little stake can build stake *through* service, and no single large node can dominate the factor and re-centralize delegation.
- **Empirical calibration.** The factor's weight (how much a unit of service is worth next to a unit of publishing) is set from live market data — which is exactly why the app-layer subscription rail ships first.

## Item 2 — Reforming PCAs → Conviction Accounts

Publisher Conviction Accounts are reformed in name and in function. The word "publisher" is dropped — the account is no longer publishing-only — leaving **Conviction Accounts**, which also matches the vocabulary V10 already uses for staking conviction. One conviction balance carries **three spend classes** — **publish / query / infer** — with a per-class meter and a holder-chosen percentage that says how much of the allowance may fund services. Conviction multipliers apply the same way to all three classes: commitment is commitment, whatever it is spent on. Service spend follows the same rules as subscriptions: spent is spent, no refunds.

Migration principles: no balance loss for any existing PCA; the service share defaults to 0% (strictly opt-in); an account that never opts in behaves exactly as before.

## Item 3 — Epoch-pooled service emissions

The settlement architecture Items 1 and 2 plug into. Subscription and allowance payments go into protocol-held pools — never into a seller's wallet, so the custody failure we actually observed in testing (a buyer's funds accidentally mixed into a seller's operations wallet) becomes impossible by construction. Once per epoch, the pool pays out to serving nodes in proportion to verified service. One pooled distribution per epoch spreads the gas cost across the entire network, which structurally removes the "fees are 1,800× the earnings" problem.

**One river, two streams.** Publishing fees and service fees flow through the *same* distribution machinery and appear in the *same* formula — but each factor can only pay out what its own stream brought in. Nodes see one reward with two ways to earn it, while the rule that makes wash-trading pointless (service rewards ≤ service payments in) stays intact. A single undivided pool would quietly break that rule: a self-dealer's subscription could capture a slice of publishing revenue and come out ahead.

**Unused allowances are why pooling matters.** If payouts were purely pass-through, a node would roughly get back what its own buyers paid. The no-refund model changes that: a buyer who used 10% of their allowance still paid 100% into the pool. That unspent value is the network's shared margin, and the Query/Inference Factor is the rule for sharing it.

**Epoch mechanics.** Each epoch: payments accumulate in the pool contract → serving nodes submit statement-backed claims (buyer-cosigned totals, Merkle-rooted, batched to save gas) → a challenge window, during which any claim can be tested against the call logs both sides keep → one distribution. The evidence layer uses the network itself: **statements are published as Knowledge Assets** in a curated Context Graph — the reward system storing its own proofs on the knowledge layer it rewards. Open parameters: who pays claim gas, how long the challenge window is, and whether payouts stay value-weighted (a node asking 7 μ/token earns factor faster than one asking 2 μ — defensible, since its buyers really paid that in, but a choice governance should see).

## Item 4 — Value capture: the operator fee, possibly split

The loop's last step is how operators turn factor performance into income, and it works today unchanged: adjust the operator fee, let delegators judge. One question the RFC must answer: publishing rewards are earned mostly by *capital* (delegators' stake), while service rewards are earned mostly by *work and hardware* (the operator's GPU, electricity, and time). A single fee misprices one of them — an operator who buys an H100 should not have to raise the fee on delegators' publishing yield to pay for it. A **dual fee** (a publishing-fee % and a service-fee %) is a small protocol change that lets operators price

hardware honestly and gives delegators a clearer signal when comparing nodes. The RFC carries this as a decision-required section, even if the answer ends up "one fee, for simplicity."

### Adjacent outstanding (tracked, not RFC-gated)

Hardware root of trust for private inference (SEV-SNP and GPU confidential computing; the encryption envelope is verified, the enclave is still a labeled simulation — see *Open questions* below) · production fiat rails, waiting on money-transmission/KYC legal review · packaging the module as a versioned entry in the **community integrations registry** (dkg integration install) against released runtimes, then upstreaming.

### Stages

| Stage                      | What happens                                                                                                                                                                                                                                                                                                                                                                                                             | Done when                                    |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|
| 0 • now                    | Prototype 4 finishes on mainnet; the final evidence report draws the line between deploy-now mechanics and RFC items                                                                                                                                                                                                                                                                                                     | the line is published                        |
| 1 • RFC draft              | " <i>Conviction Accounts v2 &amp; the Query/Inference Factor</i> " — chapters: motivation (stake follows usefulness, grounded in mainnet evidence) · pool structure (two streams) · unused-allowance economics · stake coupling and concavity · fee structure (one or two fees) · epoch mechanics (claims, challenge window, value weighting) · activation ramp · wash-trading analysis · PCA migration · parameter plan | draft circulated to the core team            |
| 2 • calibration            | The app-layer subscription rail runs in production; collect service volumes, query-to-publish ratios, buyer-concentration data                                                                                                                                                                                                                                                                                           | parameter proposals with data behind them    |
| 3 • community & governance | RFC published; open iteration; governance decision per network norms                                                                                                                                                                                                                                                                                                                                                     | an approved parameter set                    |
| 4 • testnet                | Conviction Account v2 contracts and the formula change implemented; audits; PCA migration rehearsed on forked state                                                                                                                                                                                                                                                                                                      | audited; migration proven lossless           |
| 5 • mainnet, phased        | Conviction Account spend classes activate first (small change); the Query/Inference Factor second, its weight <b>ramping up over a governance-set schedule of epochs</b> , with monitoring and adjustable parameters                                                                                                                                                                                                     | both live; ramp complete; weights governable |

### Risks, named

- **Oracle integrity** — the aggregation of signed statements must always remain challengeable against the underlying logs.
- **Wash-trading residue** — bounded by the closed loop, watched via concentration indices.
- **Cross-subsidy between streams** — prevented by giving each factor its own payout budget.

- **Delegation piling onto few nodes** — damped by the concave factor curve, and monitored.
- **Governance timing** — not the team's to promise.
- **Audit capacity** — gates Stage 4.
- **PCA migration** — must be boring. Any excitement there is a defect.

## Success, measured

Share of node revenue from services vs. publishing · number of distinct buyer-seller pairs with statement-verified volume · concentration index trending down · Conviction Account holders opting into service spending · **share of delegated stake sitting on nodes that actually serve — the alignment property, measured** · and the number that started all of this: minutes from a fresh node to a first metered completion, staying under one as the network grows.

## Open questions & challenges — trusted execution and privacy

Where this stands: the encryption envelope (HPKE) is implemented and verified byte-for-byte against RFC 9180's official test vectors, but the secure enclave behind it is still a labeled software simulation — every artifact carries `simulated: true`, and the buyer's verifier refuses simulated quotes by default. That discipline is the migration path: no privacy claim ships that the verifier cannot check.

Why trusted execution environments (TEEs) matter here: they turn privacy **policies into privacy proofs**. Today a seller can promise "we keep no logs"; nothing makes the promise checkable. A hardware-attested enclave makes it structural — and it protects both sides at once: the buyer's prompts from the seller-operator, and the seller's proprietary model weights from everyone (the seller proves the hash of weights it never reveals). That would add a third provenance class above the current two —  **enclave-attested**, over  weights-pinned, over  upstream-claimed — with the attestation as a queryable property of the offer that the market can price.

The staged path:

1. **Encryption everywhere first** (no special hardware): end-to-end HPKE on request and response bodies over *both* transports — most urgently the lane, where bodies currently travel as gossiped SWM messages readable by Context Graph peers. After this step, relaying peers carry ciphertext and routing data only.
2. **CPU enclaves** (SEV-SNP / TDX): the metering core and small CPU-served models inside attested virtual machines. The buyer's node verifies the attestation against pinned vendor root keys, ties it to the session at quote time, and references it **once per statement — never per message**, matching the cost discipline of everything else.
3. **GPU confidential computing** (H100-class): the only path that covers real inference, because a CPU enclave cannot attest to what the GPU executed.

Open questions, stated plainly:

- **The hardware floor.** GPU confidential computing is where inference actually lives, yet almost no current node operator owns such hardware — and Apple Silicon (two of the three machines in our own runs) offers no third-party enclave path at all. Does  start as a cloud-heavy tier, will the market pay its premium, and is that concentration acceptable for a decentralization project?

- **Honest threat model.** What does  defend against — a curious operator, or a physical attacker with the TEE side-channel literature in hand? The class label must say exactly what it claims, and no more.
- **Attestation lifecycle.** Keeping attestations fresh per session; surviving vendor key rotations and revocations without breaking live offers; who maintains the pinned root keys.
- **Secret weights vs. verification.** An enclave lets a seller prove the hash of hidden weights — but then who vouches that the hash corresponds to the model quality claimed? Reputation may have to carry what inspection no longer can.
- **What disputes reveal.** A contested statement is resolved by re-counting sampled calls, which exposes their content. To whom, under what selective-disclosure rules — and does third-party arbitration ever justify it?
- **Metadata.** Encryption hides message bodies, not sizes, timing, or the fact that two nodes trade at all. Padding and batching buy protection at the cost of latency. How much metadata privacy does the lane owe its users?
- **The  floor.** A resold upstream can never be more private than the upstream itself; the offer must state that floor, and  branding elsewhere in the catalog must not be allowed to blur it.
- **Law.** Privacy engineering meets data-protection law (GDPR first). Like fiat and KYC: flagged for legal review, not designed around.

---

*Prepared from the v1–v3.5 evidence reports and the Prototype 4 mainnet runs. If a mechanism is missing here, it is scheduled, not forgotten.*